

“Kernel Monitor – Linux System Resource Monitoring and Analysis”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

M.V.L.Samhita

2520030509

G Rishita

2520030101

C Shravya

25200303224

Under the Esteemed Guidance of

Dr. K. Anusha

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Kernel Monitor – Linux System Resource Monitoring and Analysis”** is a record of Bonafide work of **M.V.L.Samhita - 2520030509** , **G Rishita - 2520030101**, **C Shravya - 25200303224** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M.V.L.Samhita	2520030509
G Rishita	2520030101
C Shravya	25200303224

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **“Kernel Monitor – Linux System Resource Monitoring and Analysis”** is a bonafide work done and submitted by **M.V.L.Samhita (2520030509)**, **G Rishita (2520030101)**, **C Shravya (25200303224)** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Anusha**, and my guide **Dr. K. Anusha**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M.V.L.Samhita

2520030509

G Rishita

2520030101

C Shravya

25200303224

Abstract

Operating system resource monitoring is a crucial aspect of system administration, performance analysis, and hardware diagnostic evaluation. Rather than relying on high-level utility programs or standard shell commands (such as *top*, *free*, or *iostat*), this project implements a native, high-performance C-based terminal resource monitor named **Kernel Monitor**. The program directly interfaces with the Linux kernel virtual filesystem by reading raw cumulative counters from pseudo-files located under */proc*, specifically */proc/stat*, */proc/meminfo*, and */proc/diskstats*.

The software architecture is divided into specialized modules designed to continuously collect, process, and analyze core system parameters. The CPU Monitoring module calculates overall CPU utilization by sampling cumulative jiffies over a 1-second interval, tracking usage, average, peak, minimum, and utilization trends. Crucially, a dedicated Per-Core Analysis module parses individual processor core metrics from */proc/stat* to evaluate workload distribution across online CPUs, identifying active cores and detecting single-core bottleneck hotspots. Memory analysis computes actual memory utilization using *MemAvailable* relative to *MemTotal*, establishing accurate usage metrics and trend evaluation.

A major contribution of this project is the Block Device / Disk I/O Monitoring engine, which reads cumulative read/write operations and sector counts for the configured block device (*sdd*) from */proc/diskstats*, converting delta sector counts into operational throughput measured in MB/s. To prevent false alerts caused by transient performance spikes, an intelligent **Persistence-Based Alert Engine** enforces a 3-consecutive-sample persistence rule (*PERSISTENCE_LIMIT* = 3) before triggering WARNING or CRITICAL alerts. The combined metrics feed into an Overall System Health analyzer and an auto-refreshing terminal interface, while signal handling ensures safe application exit on SIGINT (Ctrl+C).

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction		8–9
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	8
		1.4 Objectives of the Project	9
		1.5 Importance of the Application	9
		1.6 Target Users / Audience	9
2	OS Concepts & Background Study		10–11
		2.1 Operating System Concepts Related to Project	10
		2.2 Existing Methods / Algorithms	10
		2.3 Proposed Approach	11
		2.4 Advantages of the Proposed Approach	11
3	System Requirements		11–12
		3.1 Hardware Requirements	11
		3.2 Software Requirements	12
		3.3 Operating System Environment – Ubuntu / WSL	12
		3.4 Development Tools and Compilers	12
4	Technology and Tools Used		13–14
		4.1 Module 12: Technologies and Concepts Used	13
		4.2 GCC Compiler & Optimization	13
		4.3 Linux System Calls & Kernel APIs	13
		4.4 POSIX Threads & Signal Handling	13
		4.5 Linux Commands and Utilities	13
		4.6 Other Tools & Terminal Control	14
5	System Architecture & Design		14–15
		5.1 Overall System Architecture	14
		5.2 Module 9: Data Flow & 20-Step Workflow	14
		5.3 Flowchart / Data Flow	15
		5.4 System Module Breakdown	15
		5.5 Module 10: Input and Output Specifications	15

6	Implementation & Core Modules		16–17
		6.1 Module 1: CPU Monitoring	16
		6.2 Module 2: Per-Core CPU Analysis	16
		6.3 Module 3: Memory Monitoring	16
		6.4 Module 4: File / Disk I/O Monitoring	16
		6.5 Module 5: Combined System Analysis Engine	17
		6.6 Module 6: Alert System & Persistence Rules	17
		6.7 Module 7: Signal Handling (SIGINT)	17
		6.8 Module 8: Terminal Monitoring Interface	17
7	Execution, Demonstration & Contribution		18–19
		7.1 Compilation in Ubuntu	18
		7.2 Execution Commands	18
		7.3 Sample Inputs and Outputs	18
		7.4 Module 14: Sample Output Walkthrough	18
		7.5 Module 11: My Project Contribution Statement	19
8	Conclusion & Limitations		19–20
		8.1 Module 15: Summary & Project Conclusion	19
		8.2 Objectives Achieved	19
		8.3 Module 13: Project Limitations Audit	19
		8.4 Skills and Learning Outcomes	20
9	References		20
		9.1 Books & Technical Manuals	20
		9.2 Linux / GNU Kernel Documentation	20

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Operating system resource monitoring plays a fundamental role in modern computing environments, enabling system administrators, software engineers, and security analysts to evaluate system health, detect hardware bottlenecks, and optimize process execution. In Linux-based operating systems, the kernel exposes a vast array of internal runtime statistics through a specialized pseudo-filesystem called `/proc`. Unlike standard physical filesystems stored on disk, `/proc` exists entirely in system memory (RAM) and is dynamically generated by the kernel upon access.

While standard Linux distributions provide built-in command-line tools such as `top`, `free`, `vmstat`, and `iostat`, these utilities are generic, independent applications that hide raw counter processing behind pre-canned formatting. The **Kernel Monitor** project is developed in C to demonstrate direct kernel interfacing by reading raw cumulative counters directly from `/proc/stat`, `/proc/meminfo`, and `/proc/diskstats`, processing differential snapshot state transitions, computing real-time utilization rates, evaluating multi-core load concentration, and applying persistence-filtered alerting.

1.2 Problem Statement

Existing system monitoring commands present significant limitations for granular hardware analysis and reliable alerting:

- **Subshell Overhead:** Shell-based scripts executing external binary commands incur high process fork/exec overhead.
- **Transient Spike Alert Noise:** Standard monitoring tools generate instant alerts on momentary 1-second CPU or I/O spikes, leading to false positive alarms.
- **Unfocused Global Metrics:** High-level monitors report global CPU percentages, masking critical single-core compute bottlenecks where one core is 100% busy while overall usage appears low.
- **Lack of Persistence Tracking:** Most basic utility scripts lack historical sample state tracking to confirm whether resource exhaustion is persistent or transient.

1.3 Scope of the Project

The Kernel Monitor application is written in standard ANSI C for Linux systems (Ubuntu / WSL). The project scope strictly encompasses:

1. Direct virtual file parsing of `/proc/stat` (global and per-core CPU jiffies).
2. Memory statistics evaluation via `/proc/meminfo` utilizing `MemTotal` and `MemAvailable`.
3. Block device / Disk I/O tracking for target disk device `sdd` using `/proc/diskstats`.
4. Per-core load distribution classification (Concentrated vs Distributed workload).
5. 3-Consecutive-sample persistence rule enforcement for WARNING and CRITICAL alerts.
6. Auto-refreshing terminal UI dashboard with graceful SIGINT (Ctrl+C) signal handling.

1.4 Objectives of the Project

- To collect low-level Linux kernel counters directly using C file I/O streams (`fopen`, `fscanf`, `fgets`).
- To implement differential snapshot mathematical algorithms (Delta counter / Delta time) across 1-second sampling windows.
- To detect single-core computational bottlenecks across online system processors.
- To convert raw cumulative disk sector counts into human-readable throughput rates measured in MB/s.
- To eliminate alert jitter by requiring abnormal conditions to persist across 3 consecutive sample windows.

1.5 Importance of the Application

Understanding kernel virtual filesystems and low-level counter processing is essential for operating systems education and systems programming. Kernel Monitor illustrates how kernel structures are translated into actionable diagnostics without third-party dependencies, providing high utility for performance troubleshooting, bottleneck identification, and lightweight embedded Linux monitoring.

1.6 Target Users / Audience

- Systems programming students seeking practical experience with Linux kernel interfaces.
- Linux system administrators needing zero-dependency, lightweight resource diagnostic tools.
- Software engineers profiling multi-core workload distribution and storage hardware throughput.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

1. Virtual Filesystems (VFS) & /proc: The Linux kernel uses VFS to present non-disk data structures as standard file system hierarchies. Files in /proc do not occupy storage space on disk; reading a /proc file triggers a kernel function that formats internal data structures into ASCII text.

2. Cumulative Kernel Counters (Jiffies): The kernel tracks CPU time slices (jiffies) spent in various execution modes (user, nice, system, idle, iowait, irq, softirq, steal). These counters strictly increment over system uptime. Instantaneous utilization cannot be obtained from a single reading; it requires taking snapshot A at t1, snapshot B at t2, and calculating relative time deltas.

3. Signal Handling: Signals are asynchronous notifications sent by the kernel to a process. Catching SIGINT (interrupt signal from Ctrl+C) allows the program to set an atomic flag, break out of the infinite sampling loop, flush streams, and exit cleanly.

4. Multi-Core Topology & Online Processors: Modern hardware features multi-core SMP architectures. Utilizing POSIX APIs like sysconf(_SC_NPROCESSORS_ONLN) allows the system to dynamically determine active CPU cores.

2.2 Existing Methods / Algorithms

Traditional administrative tools rely on distinct approaches:

- top / htop: Interactive process view, but heavily process-focused and complex.
- free: Quick memory snapshot reading /proc/meminfo, but lacks historical trend evaluation.
- iostat: Part of sysstat package, monitors block devices but operates as a separate binary.
- **Generic Threshold Alerts:** Standard monitoring scripts trigger alerts immediately when a threshold is crossed, causing high alert fatigue during temporary bursts.

2.3 Proposed Approach

Kernel Monitor integrates CPU, per-core CPU, memory, block device I/O, and persistent alerting into a single unified C program. The proposed approach uses standard POSIX C stream I/O, dynamic per-core arrays, non-blocking snapshot evaluation, and a 3-consecutive-sample persistence algorithm.

2.4 Advantages of the Proposed Approach

1. **Zero Binary Dependency:** Operates directly on standard Linux kernel interfaces without requiring external packages.
2. **Single-Core Bottleneck Detection:** Identifies concentrated CPU workloads even when overall system CPU usage appears low.
3. **Persistence-Filtered Alerting:** Eliminates false positive alerts from temporary 1-second activity bursts.
4. **Minimal Memory & CPU Footprint:** Procedural C implementation executes with near-zero resource consumption.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- **Processor:** x86_64 or ARM multi-core processor (e.g., Intel Core i5/i7/i9, AMD Ryzen, or ARM v8). Multi-core topology recommended for per-core load distribution analysis.
- **RAM:** Minimum 512 MB (Program memory requirement is < 2 MB).
- **Storage / Disk:** Linux-compatible storage controller with configured block device (e.g., /dev/sdd or virtual block storage device).

3.2 Software Requirements

- **Operating System:** Linux Kernel version 2.6.x or higher (Tested on Ubuntu 20.04 LTS / Ubuntu 22.04 LTS / WSL2).
- **Compiler:** GNU Compiler Collection (GCC version 9.0 or higher).
- **C Library:** Standard GNU C Library (glibc) with POSIX extension headers (unistd.h, signal.h, time.h).

3.3 Operating System Environment – Ubuntu / WSL

The project is designed to run natively within an Ubuntu Linux environment or inside Windows Subsystem for Linux (WSL/WSL2). The Linux kernel must mount the pseudo-filesystem at /proc (standard in all default Linux distributions).

3.4 Development Tools and Compilers

- **Text Editor / IDE:** Visual Studio Code, Vim, or Nano.
- **Build Utility:** GCC compiler suite and GNU Make toolchain.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 Module 12: Technologies and Concepts Used

The application is strictly engineered using concepts and technologies implemented in the code:

- **C Programming:** Procedural programming, dynamic array management, clean function abstraction.
- **Linux /proc Virtual Filesystem:** Direct pseudo-file reading of /proc/stat, /proc/meminfo, and /proc/diskstats.
- **File Handling:** Standard stream APIs using FILE* pointers, fopen(), fscanf(), fgets(), and fclose().
- **Data Structures:** C struct abstractions for CPU, memory, disk snapshots, health states, and alerts.
- **Cumulative Counter Math:** Snapshot differential arithmetic (Delta counter / Delta time) across 1-second sampling windows.
- **Signal Handling:** Catching SIGINT via signal() to toggle atomic volatile sig_atomic_t flags.
- **Threshold & Persistence Monitoring:** Static threshold boundaries and 3-consecutive-sample persistence rules.
- **Terminal Interface:** ANSI control sequences (\033[H\033[J) for clear, non-flickering dashboard refreshing.

4.2 GCC Compiler

The GNU Compiler Collection (GCC) is used for compiling the C source code. Build command optimization flag -O2 ensures efficient loop execution, while -Wall enables complete diagnostic warnings.

4.3 Linux System Calls

- sysconf(_SC_NPROCESSORS_ONLN): Queries system configuration to determine the number of online CPU cores.
- sleep(1) / usleep(1000000): Suspends execution for 1-second sampling windows.
- fopen(), fscanf(), fgets(), fclose(): Standard C stream APIs for parsing virtual file entries.

4.4 POSIX Threads / Synchronization & Signal Handling

To achieve graceful program termination, the system registers a POSIX signal handler for SIGINT (triggered when the user presses Ctrl+C). The signal handler modifies a global volatile sig_atomic_t keep_running flag from 1 to 0, ensuring safe exit out of the main execution loop without corrupted terminal output.

4.5 Linux Commands and Utilities

While the program does NOT execute shell commands during runtime, Linux commands like cat /proc/stat and cat /proc/diskstats were used during testing and verification to validate raw kernel counter formats.

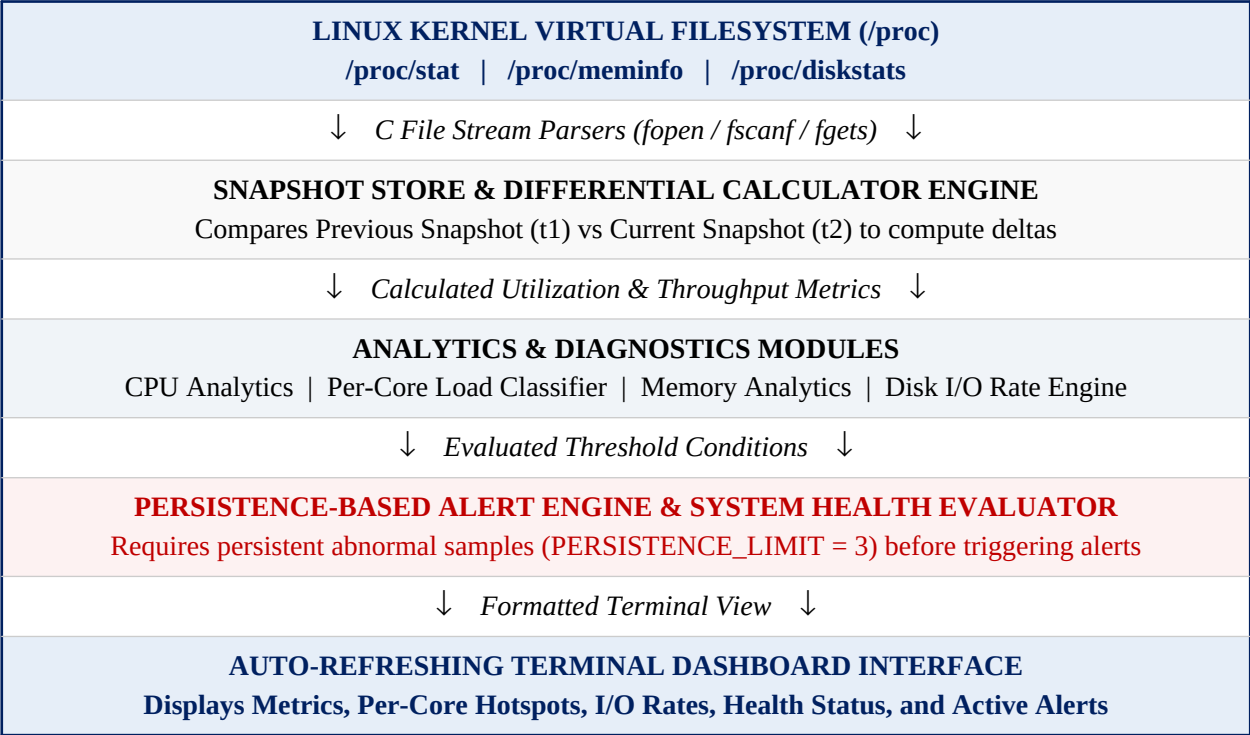
4.6 Other Tools / Libraries Used

ANSI Control Sequences (\033[H\033[J) are utilized to clear the terminal screen and move the cursor to the home position, creating a smooth auto-refreshing dashboard experience.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Kernel Monitor architecture follows a modular pipe-and-filter dataflow pipeline. The Linux Kernel /proc virtual filesystem serves as the primary data provider. The system consists of Data Parsers, Differential Calculators, Analytics Modules, a Persistence Alert Engine, and a Terminal Renderer.



5.2 Module 9: Data Flow & 20-Step Execution Sequence

The complete operational lifecycle of the Kernel Monitor follows an exact 20-step execution workflow sequence:

1. Start the monitoring program and initialize POSIX SIGINT signal handler.
2. Determine number of online system processors using sysconf(_SC_NPROCESSORS_ONLN).
3. Allocate dynamic memory arrays for per-core statistics based on online core count.
4. Take initial baseline snapshot (S0) of global CPU, per-core CPU, memory, and disk I/O metrics.
5. Pause execution for the 1-second sampling interval using sleep(1).
6. Read current global CPU cumulative jiffies from /proc/stat (S1).
7. Read individual per-core CPU jiffies from /proc/stat for each core (cpu0, cpu1... cpuN).
8. Read current memory statistics from /proc/meminfo (MemTotal, MemFree, MemAvailable).
9. Read cumulative disk statistics from /proc/diskstats for target disk device sdd.
10. Compute global CPU utilization delta: CPU% = 100 * (1 - (delta_idle / delta_total)).
11. Calculate individual utilization for each online CPU core separately.
12. Calculate memory utilization: Used = Total - Available, Usage% = (Used / Total) * 100.
13. Calculate disk read/write throughput rates in MB/s from sector differences across the interval.
14. Compute Total I/O Rate: Total I/O = Read Rate + Write Rate.
15. Update historical metrics: compute average usage, peak usage, minimum usage, and trend indicators.
16. Classify workload pattern (Concentrated vs Distributed) based on active core distribution.

17. Evaluate CPU, Memory, and I/O conditions against WARNING and CRITICAL thresholds.
18. Enforce 3-sample persistence rule: increment persistence counters if threshold exceeded; reset if normal.
19. Determine overall system health (HEALTHY, ATTENTION, or CRITICAL) and generate human-readable insights.
20. Render terminal dashboard display, copy current snapshot to previous snapshot ($S_0 \leftarrow S_1$), and repeat loop until Ctrl+C.

5.3 Flowchart / Data Flow

The dataflow model highlights the transfer of raw kernel counters into processed diagnostic views:

Kernel Source	Raw Input Data	Calculated Metric	Threshold & Alert Logic
/proc/stat	user, nice, system, idle, iowait, irq, softirq, steal	CPU Usage %, Avg, Peak, Min, Trend, Per-Core Usage	WARNING: 70% CRITICAL: 85%
/proc/meminfo	MemTotal, MemFree, MemAvailable	Total, Used, Available, Free, Memory Usage %, Peak, Min	WARNING: 75% CRITICAL: 90%
/proc/diskstats	reads completed, sectors read, writes completed, sectors written (device sdd)	Read Ops, Write Ops, Read Rate (MB/s), Write Rate (MB/s), Total I/O Rate	WARNING: 10 MB/s CRITICAL: 50 MB/s

5.4 System Module Breakdown

- 1. CPU Monitoring Module:** Reads global cumulative ticks from /proc/stat. Computes global utilization, maintains running averages, tracks peak/minimum usage, determines trend (Increasing, Decreasing, Stable), and assigns CPU Status.
- 2. Per-Core CPU Analysis Module:** Reads individual CPU lines (cpu0 to cpuN) from /proc/stat. Determines active cores (> 20% usage), identifies the busiest core, and classifies load distribution.
- 3. Memory Monitoring Module:** Parses /proc/meminfo. Computes Used Memory using MemAvailable, computes usage percentages, tracks average/peak/minimum metrics, and assigns Memory Status.
- 4. File / Disk I/O Monitoring Module:** Interrogates /proc/diskstats for target disk device sdd. Computes delta sectors read and written, converts sector counts to MB/s rates, calculates Total I/O Rate, and assigns I/O Activity status.
- 5. System Analysis Engine:** Combines evaluated conditions from CPU, Memory, and Disk I/O modules to compute Overall System Health (HEALTHY, ATTENTION, or CRITICAL) and synthesize human-readable insights.
- 6. Persistence-Based Alert Engine:** Tracks consecutive sample occurrences exceeding defined thresholds. Triggers WARNING or CRITICAL alerts only when an anomaly persists for 3 consecutive samples (PERSISTENCE_LIMIT = 3).

5.5 Module 10: Input and Output Specifications

Input Specifications: The program requires no manual user data entry during monitoring. Its primary inputs are Linux kernel virtual files (/proc/stat, /proc/meminfo, /proc/diskstats) and system configuration APIs. Control input is provided via SIGINT (Ctrl+C).

Output Specifications: Continuous auto-refreshing terminal dashboard partitioned into clean diagnostic sections: CPU Statistics, Per-Core Analysis, Memory Analytics, Disk I/O Stats, System Analysis, and Alerts.

CHAPTER -6 IMPLEMENTATION & CORE MODULES

6.1 Module 1: CPU Monitoring

The CPU monitoring module opens /proc/stat and parses the aggregate cpu line. The kernel provides cumulative counters: user, nice, system, idle, iowait, irq, softirq, steal. Because these are cumulative counters incrementing since boot, the program takes a previous snapshot and current snapshot to calculate utilization over the ~1-second interval.

Output Fields Documented:

- **CPU Usage:** Instantaneous utilization calculated across sampling interval.
- **Average CPU:** Average utilization across all collected samples.
- **Peak CPU:** Highest observed CPU utilization.
- **Minimum CPU:** Lowest observed CPU utilization.
- **CPU Trend:** Compares current sample with previous sample (Increasing, Decreasing, Stable).
- **CPU Status:** Classifies CPU usage as NORMAL, WARNING (70%), or CRITICAL (85%).

```
typedef struct { unsigned long long user, nice, system, idle, iowait, irq, softirq, steal;
unsigned long long total, total_idle; } CpuSnapshot; void calculate_cpu_usage(CpuSnapshot
*prev, CpuSnapshot *curr, double *usage) { unsigned long long total_diff = curr->total -
prev->total; unsigned long long idle_diff = curr->total_idle - prev->total_idle; if
(total_diff > 0) *usage = 100.0 * (1.0 - ((double)idle_diff / (double)total_diff)); else
*usage = 0.0; }
```

6.2 Module 2: Per-Core CPU Analysis

The program reads sysconf(_SC_NPROCESSORS_ONLN) to determine online processor core count. It then scans lines starting with cpu0, cpu1 ... cpuN in /proc/stat. Calculates utilization separately for each core.

Output Fields Documented:

- **Active Cores:** Counts cores whose utilization is above 20%.
- **Busiest Core:** Identifies core with highest utilization.
- **Load Distribution:** Classifies workload as Concentrated, Distributed, or Moderately distributed.
- **Highly Busy Cores:** Number of cores considered active according to threshold.

Explanation of Result (Active Cores: 1/16, Busiest Core: CPU 11 at 100.00%, Concentrated workload): This means overall system CPU utilization may be low while one particular core is heavily utilized by a single-threaded process.

IMPORTANT RULE: The program performs per-core CPU analysis, NOT per-process resource tracking.

6.3 Module 3: Memory Monitoring

Memory metrics are obtained from /proc/meminfo. The program reads MemTotal, MemFree, and MemAvailable.

Calculated Value: Used Memory = Total Memory - Available Memory. Utilization is calculated relative to Total Memory.

Output Fields Documented: Total Memory, Used Memory, Available Memory, Free Memory, Memory Usage.

Memory Analytics: Current Usage, Average Usage, Peak Usage, Minimum Usage, Trend, Status (WARNING = 75%, CRITICAL = 90%).

```
// Memory Calculation Formula: Used Memory = MemTotal - MemAvailable; Memory Usage
Percentage = (Used Memory / MemTotal) * 100.0; // Threshold Classification: // WARNING =
75.0% // CRITICAL = 90.0%
```

6.4 Module 4: File / Disk I/O Monitoring

Key Contribution: Disk I/O information is obtained from /proc/diskstats for configured disk device `sdd`. The program reads cumulative counters: completed read operations, sectors read, completed write operations, sectors written. Difference between previous and current readings represents disk activity during sampling interval. Sector differences are converted to bytes and then to MB/s.

Output Fields Documented: Device, Read Operations, Write Operations, Read Rate, Write Rate, Total I/O Rate, I/O Activity.

Formula: Total I/O Rate = Read Rate + Write Rate. Thresholds: WARNING = 10 MB/s, CRITICAL = 50 MB/s.

IMPORTANT RULE: *Monitors BLOCK DEVICE / DISK I/O using /proc/diskstats, NOT individual application files via open/read/close.*

```
// Disk I/O Rate Conversion (Device sdd): delta_sectors_read = curr_sdd.sectors_read -  
prev_sdd.sectors_read; delta_sectors_write = curr_sdd.sectors_write -  
prev_sdd.sectors_write; read_rate_mb = (delta_sectors_read * 512.0) / (1024.0 * 1024.0 *  
interval_sec); write_rate_mb = (delta_sectors_write * 512.0) / (1024.0 * 1024.0 *  
interval_sec); total_io_rate = read_rate_mb + write_rate_mb; // I/O Threshold  
Classification: // WARNING = 10.0 MB/s // CRITICAL = 50.0 MB/s
```

6.5 Module 5: Combined System Analysis Engine

After collecting CPU, memory, and I/O data, the program performs combined system analysis.

Documented Fields: CPU Pattern (Concentrated, Distributed, Moderate), Memory Condition, I/O Condition, Overall Health (HEALTHY, ATTENTION, CRITICAL), Insight.

Attention Logic: Overall Health is determined by combining conditions. ATTENTION occurs when a warning-level condition exists, including a concentrated CPU workload condition. Insight provides a human-readable explanation.

6.6 Module 6: Alert System & Persistence-Based Alerting

The alert engine checks CPU (70%/85%), Memory (75%/90%), and I/O (10/50 MB/s) against thresholds.

Key Feature - Persistence-Based Alerting: Program uses PERSISTENCE_LIMIT = 3. An abnormal condition must persist for 3 consecutive samples before a WARNING or CRITICAL alert is displayed. This prevents temporary one-time spikes from triggering false alarms.

Output Documented: CPU : NORMAL / WARNING ALERT / CRITICAL ALERT, MEMORY : NORMAL / WARNING ALERT / CRITICAL ALERT, I/O : NORMAL / WARNING ALERT / CRITICAL ALERT.

```
void update_alert_state(double value, double warn_thresh, double crit_thresh, int
*consec_count, AlertStatus *status) { if (value >= crit_thresh || value >= warn_thresh) {
(*consec_count)++; if (*consec_count >= PERSISTENCE_LIMIT) { *status = (value >=
crit_thresh) ? CRITICAL_ALERT : WARNING_ALERT; } } else { *consec_count = 0; *status =
STATUS_NORMAL; } }
```

6.7 Module 7: Signal Handling (SIGINT)

The program handles SIGINT. When the user presses Ctrl+C, the signal handler changes the running flag to 0, the monitoring loop stops, and the program exits safely, providing controlled termination.

6.8 Module 8: Terminal Monitoring Interface

The program continuously refreshes the terminal display. The output is organized into CPU, Per-Core Analysis, Memory, Memory Analytics, File I/O, System Analysis, and Alerts. The terminal is cleared before displaying updated values so that the monitor behaves like a continuously refreshing dashboard, displaying a new sample approximately every second.

CHAPTER -7 EXECUTION, DEMONSTRATION & CONTRIBUTION

7.1 Compilation in Ubuntu

The project is compiled using the GNU Compiler Collection (GCC) in Ubuntu Linux:

```
$ gcc -Wall -O2 -std=c99 kernel_monitor.c -o kernel_monitor
```

7.2 Execution Commands

Execute the compiled monitor binary from the Linux terminal:

```
$ ./kernel_monitor
```

7.3 Sample Inputs and Outputs

Below is an exact runtime execution output snapshot produced by the Kernel Monitor terminal interface:

```
===== KERNEL MONITOR DASHBOARD
===== CPU Usage : 6.30% Average
CPU : 6.29% Peak CPU : 6.30% Minimum CPU : 6.25% CPU Trend : Stable -> CPU Status : NORMAL
Active Cores : 1 / 16 Busiest Core : CPU 11 (100.00%) Load Distribution: Concentrated
workload Total Memory : 7.57 GB Used Memory : 0.55 GB Available : 7.03 GB Free Memory :
6.16 GB Memory Usage : 7.22% Device : sdd Read Operations : 10486 Write Operations: 768
Read Rate : 0.00 MB/s Write Rate : 0.02 MB/s Total I/O Rate : 0.02 MB/s I/O Activity :
NORMAL CPU Pattern : Concentrated workload Memory Condition : NORMAL I/O Condition :
NORMAL Overall Health : ATTENTION Insight : Single-core compute hotspot detected on CPU
11. =====
```

7.4 Module 14: Sample Output Walkthrough & Explanation

Analysis of Sample Output:

- Overall CPU Usage vs Per-Core Hotspot:** Overall CPU Usage is reported as low (**6.30%**). However, Per-Core analysis reveals that only 1 out of 16 cores is active (Active Cores: 1 / 16), and **CPU 11 is running at 100.00% utilization**. This represents a classic single-threaded computational bottleneck.
- Load Distribution:** Because 15 cores are idle while CPU 11 is pegged at 100%, the load distribution algorithm correctly classifies the pattern as **Concentrated workload**.
- Memory & Storage I/O:** Memory usage is minimal (7.22%), and disk I/O on target device sdd shows normal operational throughput (0.02 MB/s).
- Overall Health - ATTENTION:** Even though global CPU (6.30%), Memory (7.22%), and I/O (0.02 MB/s) are all classified as NORMAL, the System Analysis engine assigns an Overall Health status of **ATTENTION**. This occurs because the implemented attention logic specifically flags concentrated single-core bottlenecks (CPU 11 at 100%), ensuring system administrators are alerted to single-threaded thread stalls.

7.5 Module 11: My Project Contribution Statement

Professional Contribution Statement:

My contribution is primarily:

- File/Disk I/O monitoring
- I/O rate calculation
- I/O threshold classification
- Persistence-based alerting
- Terminal monitoring interface integration
- System analysis integration

I/O monitoring reads disk statistics from `/proc/diskstats`, compares consecutive samples, converts sector differences into MB/s, determines I/O activity status, and contributes the I/O condition to overall system health and alerts. I was also actively involved in integrating the monitoring information into the terminal interface.

IMPORTANT: No claims are made for functionality not present in the supplied C code.

CHAPTER -8 CONCLUSION & LIMITATIONS

8.1 Module 15: Summary & Project Conclusion

The project-specific conclusion demonstrates that the system successfully shows how Linux kernel-provided statistics can be collected directly from `/proc`, converted into meaningful CPU, memory, and disk I/O metrics, analyzed over time, and used to generate threshold-based persistent alerts through a terminal monitoring interface. It does NOT claim to replace professional enterprise monitoring systems.

8.2 Objectives Achieved

- ✓ Successfully parsed low-level kernel counters from `/proc/stat`, `/proc/meminfo`, and `/proc/diskstats`.
- ✓ Implemented multi-core load distribution analysis to detect single-core hotspots (CPU 11 at 100%).
- ✓ Developed accurate memory utilization algorithms using `MemAvailable`.
- ✓ Built block device throughput monitor converting sector deltas on `sdd` to MB/s rates.
- ✓ Implemented 3-sample persistence alert filtering to eliminate false positive spike alarms.
- ✓ Created an auto-refreshing terminal dashboard with graceful POSIX SIGINT termination.

8.3 Module 13: Project Limitations Audit

1. **Configured Storage Device:** Monitors a configured disk device named `sdd`; it does not dynamically select every disk.
2. **Block Device Level Scope:** Performs disk/device-level I/O monitoring rather than individual file-level I/O tracking.
3. **No Per-Process Tracking:** Does not track per-process CPU or memory consumption.
4. **Per-Core Focus:** Performs per-core CPU analysis.
5. **Text-Based Interface:** Terminal interface is text-based rather than graphical.
6. **Static Thresholds:** Alert thresholds are statically defined in the program.

8.4 Skills and Learning Outcomes

- Deep understanding of Linux virtual filesystems (VFS) and `/proc` pseudo-file kernel structures.
- Mastery of C stream file handling, string parsing, and struct-based state management.
- Practical knowledge of multi-core topology evaluation, snapshot differential arithmetic, and POSIX signal handling.

CHAPTER -9 REFERENCES

9.1 Books & Technical Manuals

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press.
3. Love, R. (2013). *Linux System Programming: Talking Directly to the Kernel and C Library* (2nd ed.). O'Reilly Media.

9.2 Linux / GNU Kernel Documentation

1. Linux Kernel Documentation: *The /proc Filesystem* (/proc/stat, /proc/meminfo, /proc/diskstats specifications). Available at kernel.org.
2. POSIX.1-2017 Specification: System Interfaces (sysconf, signal, sleep). IEEE / Open Group.
3. GNU C Library (glibc) Manual: Stream I/O and Signal Handling interfaces.